Object-Oriented Programming

Classes, Objects, Encapsulation, the 4 Pillars,
OOP vs Procedural, and C vs C++

A practical guide covering what each concept means, why it exists, how to use it well, and runnable C++ examples for each idea.

1. What Is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **data (objects)** rather than just functions and logic. Instead of writing a program as a sequence of steps that operate on separate data, OOP bundles data and the functions that operate on that data into a single unit called an **object**.

The goal is to model real-world entities (a Car, a BankAccount, an Employee) as self-contained units that manage their own state and behavior, making large programs easier to design, extend, and maintain.

Why OOP Is Needed

- **Managing complexity** — as programs grow, procedural code becomes a tangle of functions and global data that is hard to track. OOP groups related data and behavior together.
- **Reusability** — once a class is written and tested, it can be reused across projects or extended via inheritance instead of rewritten.
- **Maintainability** — changes to one object's internal logic don't ripple through the entire codebase, as long as its interface stays the same.
- **Real-world modeling** — objects map naturally to real entities, making designs more intuitive to reason about.
- **Security** — encapsulation lets objects hide internal details and expose only what's necessary.

OOP vs. Procedural Programming

Procedural programming (C, plain scripts) structures a program as a series of functions that operate on shared or passed-around data. OOP (C++, Java, Python, etc.) structures a program as a set of interacting objects that each own their data and behavior.

Aspect	Procedural	Object-Oriented
Basic unit	Function	Object (data + methods)
Data handling	Global/shared data, passed to functions	Data bound inside objects (encapsulation)
Security	Data mostly exposed, less protected	Data hidden, accessed via controlled methods
Code reuse	Copy-paste or function reuse	Inheritance, composition, polymorphism
Scalability	Harder to scale — tightly coupled logic	Easier to scale — modular, loosely coupled objects
Real-world mapping	Low — logic-centric	High — entity-centric
Change impact	A change can affect many functions	Changes are localized to a class if the interface is stable

In short: procedural programming asks *"what steps do I run?"*, while OOP asks *"what entities exist, and what can each of them do?"*

2. Class

A **class** is a blueprint or template for creating objects. It defines what data (member variables / attributes) and what behavior (member functions / methods) every object created from it will have. A class itself does not occupy memory for its data until an object is instantiated from it.

Why Classes Are Required

- They let you define a structure **once** and create many objects from it, instead of repeating variable/function declarations for every entity.
- They group related data and functions logically, matching how we think about real entities.
- They form the basis for inheritance and polymorphism — without classes, there is no type hierarchy to build on.

How to Use Classes Optimally

- Keep a class focused on **one responsibility** (Single Responsibility Principle) — a Car class should not also handle database logging.
- Keep member variables **private** and expose behavior through well-named public methods.
- Use constructors to guarantee an object always starts in a valid state.
- Prefer composition ("has-a") over deep inheritance chains when it keeps the design simpler.

Example: Defining and Using a Class in C++

```
#include <iostream>
using namespace std;

class Car {
    private:
        string brand;
        int speed;

    public:
        // Constructor
        Car(string b, int s) {
            brand = b;
            speed = s;
        }

        // Member function
        void displayInfo() {
            cout << brand << " is running at " << speed << " km/h" << endl;
        }
};

int main() {
    Car car1("Toyota", 120);   // object created from class
    Car car2("Honda", 100);

    car1.displayInfo();
    car2.displayInfo();
    return 0;
}
```

Output:

Toyota is running at 120 km/h
Honda is running at 100 km/h

3. Object

An **object** is a concrete instance of a class. If a class is the blueprint, an object is the actual building constructed from it — with its own real values stored in memory. Each object gets its own copy of the class's member variables (unless declared static), but shares the same member function code.

Why Objects Are Required

- A class alone is just a definition; you need objects to actually store and manipulate data at runtime.
- Multiple objects of the same class can exist independently, each with its own state (e.g. two Car objects with different speeds).
- Objects are how a program interacts with the outside world — they're passed around, stored in collections, and passed to functions.

How to Use Objects Optimally

- Create objects with the minimum required scope/lifetime — avoid unnecessary global objects.
- Prefer stack allocation (`Car c1(...)`) over heap allocation (`new Car(...)`) unless you specifically need dynamic lifetime, to avoid manual memory management and leaks.
- If you do use pointers/heap objects, use smart pointers (`unique_ptr`, `shared_ptr`) in modern C++ instead of raw `new`/`delete`.
- Initialize every object through a constructor so it never exists in a half-valid state.

Example: Multiple Objects, Independent State

```
class Counter {
    private:
        int count;
    public:
        Counter() { count = 0; }
        void increment() { count++; }
        int getCount() { return count; }
};

int main() {
    Counter a, b;      // two independent objects

    a.increment();
    a.increment();
    b.increment();

    cout << "a = " << a.getCount() << endl;  // a = 2
    cout << "b = " << b.getCount() << endl;  // b = 1
    return 0;
}
```

Even though a and b are built from the same class, each keeps its own independent `count` — that's the essence of an object's state.

4. Encapsulation

Encapsulation is the bundling of data (variables) and the methods that operate on that data into a single unit (a class), combined with **restricting direct access** to some of an object's internal details. In C++ this is done using access specifiers: `private`, `protected`, and `public`.

Why Encapsulation Is Required

- **Data protection** — prevents external code from putting an object into an invalid state (e.g. setting a negative age).
- **Controlled access** — you decide exactly how the internal data can be read or modified, via getters/setters or specific methods.
- **Flexibility to change internals** — you can change how data is stored internally without breaking code that uses the class, as long as the public interface stays the same.
- **Reduced coupling** — other parts of the program depend only on the public interface, not on implementation details.

How to Use Encapsulation Optimally

- Default to making member variables `private`; only make them public if there's a clear reason.
- Add validation inside setter methods (e.g. reject a negative balance) rather than trusting the caller.
- Expose only the methods a user of the class actually needs — don't leak internal helper functions as public.
- Use `const` methods for functions that only read data, so it's clear (and enforced) they don't modify state.

Example: Encapsulated Bank Account

```
class BankAccount {
    private:
        double balance;   // hidden from outside — cannot be accessed directly

    public:
        BankAccount(double initial) {
            balance = (initial >= 0) ? initial : 0;
        }

        void deposit(double amount) {
            if (amount > 0) balance += amount;
        }

        bool withdraw(double amount) {
            if (amount > 0 && amount <= balance) {
                balance -= amount;
                return true;
            }
            return false;  // reject invalid withdrawal
        }

        double getBalance() const {
            return balance;
```

```
        }
};

int main() {
    BankAccount acc(1000);
    acc.deposit(500);
    acc.withdraw(2000);          // rejected, insufficient funds
    // acc.balance = -50;        // NOT ALLOWED - balance is private

    cout << "Balance: " << acc.getBalance() << endl;  // Balance: 1500
    return 0;
}
```

Without encapsulation, any code could set `balance` directly to an invalid or negative value. With it, the class guarantees its own rules are always enforced.

5. The Four Pillars of OOP

OOP rests on four core principles. Together they let programs be modular, extensible, and safe against misuse.

1. Encapsulation

Bundling data and methods together and restricting direct access to internal state. Achieved via access specifiers (private/protected/public) and getter/setter methods. Protects object integrity.

2. Abstraction

Hiding complex implementation details and exposing only the essential features. A driver uses a steering wheel and pedals without knowing engine internals. In C++, achieved via abstract classes and interfaces (pure virtual functions).

3. Inheritance

A mechanism where a new class (derived/child) acquires properties and behavior of an existing class (base/parent). Promotes code reuse — e.g. a SportsCar class can inherit from a generic Vehicle class instead of rewriting shared logic.

4. Polymorphism

The ability of different objects to respond to the same function call in their own way. 'Poly' = many, 'morph' = forms. Achieved via function overloading (compile-time) and virtual functions / overriding (runtime).

Example: Inheritance and Polymorphism Together

```
class Shape {                 // Abstraction: defines a common interface
    public:
        virtual double area() = 0;   // pure virtual -> abstract class
};

class Circle : public Shape {        // Inheritance: reuses Shape's interface
    private:
        double radius;
    public:
        Circle(double r) { radius = r; }
        double area() override {     // Polymorphism: own implementation
            return 3.1416 * radius * radius;
        }
};

class Rectangle : public Shape {     // Inheritance
    private:
        double length, width;
    public:
```

```
        Rectangle(double l, double w) { length = l; width = w; }
        double area() override {     // Polymorphism: different implementation
            return length * width;
        }
};

int main() {
    Shape* shapes[2];
    shapes[0] = new Circle(5);
    shapes[1] = new Rectangle(4, 6);

    for (int i = 0; i < 2; i++) {
        cout << "Area: " << shapes[i]->area() << endl;
        delete shapes[i];
    }
    return 0;
}
```

Both `Circle` and `Rectangle` inherit from `Shape` and respond to the **same** `area()` call, each in its own way — this is inheritance and polymorphism working together, while `Shape` itself is an abstraction of "any shape with an area."

6. How C++ Differs from C

C++ was built as an extension of C, adding object-oriented features on top of C's procedural core. Almost all valid C code compiles as C++ (with minor differences), but C++ adds an entire additional programming model.

Aspect	C	C++
Paradigm	Procedural only	Procedural + Object-Oriented
Core unit	Functions	Classes and objects
Data & functions	Kept separate	Bundled together (encapsulation)
Inheritance	Not supported	Supported
Polymorphism	Not supported natively	Supported (overloading, overriding, virtual functions)
Data hiding	No built-in access control	private / protected / public access specifiers
Function overloading	Not supported	Supported (same name, different parameters)
Standard Library	Limited standard functions (stdio.h, etc.)	Rich Standard Template Library (STL): vectors, maps, algorithms
Memory management	malloc / free	new / delete, plus smart pointers (RAII)
Exception handling	Not supported (error codes/return values)	Supported (try / catch / throw)
Namespaces	Not supported	Supported (avoids name collisions)
Reference variables	Not supported (pointers only)	Supported (&)
Templates / generics	Not supported	Supported (generic programming)

Quick Example: The Same Idea in C vs C++

```c
/* C - procedural style: data and functions are separate */
struct Rectangle {
    float length, width;
};

float area(struct Rectangle r) {   // free function, operates on the struct
    return r.length * r.width;
}

int main() {
    struct Rectangle r = {4, 6};
    printf("%f\n", area(r));
    return 0;
}
```

```cpp
// C++ - OOP style: data and functions bundled into a class
class Rectangle {
    private:
        float length, width;
    public:
        Rectangle(float l, float w) { length = l; width = w; }
        float area() { return length * width; }   // method belongs to the object
};
```

```
int main() {
    Rectangle r(4, 6);
    cout << r.area() << endl;
    return 0;
}
```

In C, the rectangle's data and the function that computes its area are two unrelated things. In C++, `area()` belongs to the `Rectangle` object itself — this is the essence of the shift from procedural to object-oriented design.

Summary Takeaway

C++ keeps everything C can do, then layers classes, encapsulation, inheritance, and polymorphism on top — giving you the choice to write purely procedural code, purely object-oriented code, or a mix of both depending on what a given problem needs.